

MLOps Pipeline with Docker and Kubernetes

Deploying a Production-Ready PyTorch Image Classification Model

Name: Risan Raja

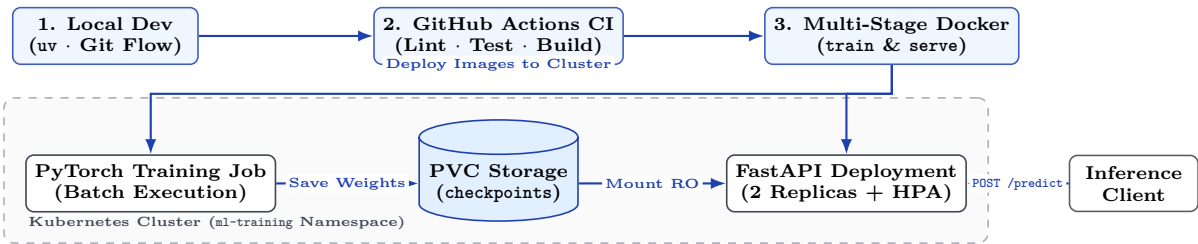
Roll Number: DA25G521

GitHub Repo: github.com/risa-raja/mlops-pytorch-pipeline

Final PR: PR #5 (develop → main)

1 Pipeline Overview & System Architecture

This technical report presents an end-to-end, production-ready Machine Learning pipeline illustrating the complete operational deployment lifecycle of a PyTorch image classification model (CIFAR-10): from local development using Git Workflows and the Astral uv package manager, to multi-stage containerized Docker builds, automated batch job scheduling for training, and autoscaled inference service deployment in Kubernetes.



2 Git Workflow & Pull Requests

This repository strictly adheres to the Git Flow branching model and the **Conventional Commits** specification:

- main**: Production release branch.
- develop**: Primary integration branch.
- feature/***: Isolated feature branches merged via reviewed Pull Requests with structured changelogs.

2.1 Pull Requests Summary

PR	Branch Target	Key Implementations & Deliverables	Status
PR #1	feature/requirements → develop	Setup Astral uv package management, pyproject.toml, lockfiles, and CPU-only PyTorch indexes.	Merged
PR #2	feature/ci → develop	Added GitHub Actions CI pipeline with Ruff linting, Pyrefly type checking, pytest unit tests, and Docker validation.	Merged
PR #3	feature/modelling → develop	Implemented PyTorch CNN model, CIFAR-10 data loaders, training loop with JSON metrics, and FastAPI serving.	Merged
PR #4	feat/kubernetes → develop	Implemented multi-stage Dockerfiles, Kubernetes Job, persistent storage claims, autoscaled Deployments, and probes.	Merged
PR #5	develop → main	Consolidated production release merging all tested infrastructure, automation scripts, and documentation.	Merged

Table 1: Summary of Git Flow Pull Requests and integration deliverables.

3 Reflection & Technical Analysis

1. What was the most difficult part of this project?

The most difficult portion of developing this pipeline was managing **persistence of data in Kubernetes asynchronous Job streams**, as well as **containerized storage provisioning**:

- **Compatibility between storage class access modes and provisioning for local containers:** The local provisioners for Kubernetes (for example Docker Desktop / Rancher local-path) can only provide a `ReadWriteOnce` access mode. Distributed clusters utilize cloud-based or NFS file systems that provide a `ReadWriteMany` access mode. Creating a checkpoint persistent volume claim that was writable by the batch Job and subsequently read-only mounted by the serving deployment required managing both the storage life cycle and isolation.
- **Probing timings & cold starts:** During initialization, the container loads the PyTorch model weights from disk into memory. By tuning `initialDelaySeconds` (15 seconds) and `periodSeconds` on the Readiness Probe, we could prevent Kubernetes from routing user-inference requests to uninitialized containers during the initial cold-start process.

2. Using Astral's uv in a Multi-Stage Build to Improve Dependencies

Utilizing Astral's `uv` within our Docker multi-stage builds (`ghcr.io/astral-sh/uv` builder + `python:3.13-slim` runtime) greatly improved our build speeds and allowed us to take advantage of Docker's layer caching. We configured an explicit PyTorch-cpu wheel index to preclude the need for multi-gigabyte CUDA binaries to bloat our image size and keep the final serving container lightweight and fast to start.

4 Summary of Pipeline Manifests & Components

Layer	File / Manifest	Operational Role
Configuration	<code>configs/training_config.yaml</code>	Declarative hyperparameters (LR, batch size, epochs, early stopping).
Containers	<code>docker/Dockerfile.train</code>	Multi-stage PyTorch training image with volume mount points.
	<code>docker/Dockerfile.serve</code>	Non-root lightweight inference container running FastAPI on port 8080.
Kubernetes	<code>k8s/namespace.yaml</code>	Dedicated <code>ml-training</code> namespace for isolated workloads.
	<code>k8s/configmap.yaml</code>	Declarative hyperparameters mounted to <code>/app/configs</code> .
	<code>k8s/pvc.yaml</code>	<code>PersistentVolumeClaim</code> for model weights persistence (<code>/app/checkpoints</code>).
	<code>k8s/training-job.yaml</code>	Batch training Job with CPU/Memory limits and GPU support readiness.
	<code>k8s/serving-deployment.yaml</code>	2-replica serving Deployment with Readiness and Liveness probes.
	<code>k8s/serving-service.yaml</code>	ClusterIP Service routing cluster port 80 to container port 8080.
	<code>k8s/hpa.yaml</code>	Horizontal Pod Autoscaler targeting 70% CPU utilization.

Table 2: Key configuration manifests, container definitions, and deployment artifacts.